

No Memory Abstraction

1 What This Topic Means

Core Definition

No memory abstraction means a program uses **physical memory addresses** directly. The address written in the instruction is the actual address used by the hardware.

The simplest possible memory model is this:

$$\text{program address} = \text{physical memory address}$$

Memory is viewed as a linear sequence of addresses:

$$0, 1, 2, 3, \dots, \text{maximum address.}$$

Each address names a memory cell, usually one byte. If a program executes the instruction below, the CPU reads physical memory address 1000.

```
MOV REGISTER1,1000
```

There is no private address space, no virtual memory, and no automatic boundary between one program and another.

Historical Context

This model was used by early mainframes, early minicomputers, and early personal computers. It still appears in simple embedded systems where all software is known in advance.

2 Why Memory Abstraction Is Needed

The Problem

If two independent programs use the same physical address, one program can overwrite the other's code or data.

Suppose two programs are loaded at the same time:

- Program P_1 stores its variable `count` at physical address 2000.
- Program P_2 stores an instruction or variable at physical address 2000.

If P_1 writes a new value to address 2000, it changes the memory cell that P_2 is also using. The result may be corrupted data, a wrong instruction, or an immediate crash.

Main Danger

No memory abstraction means no isolation. A bug in a user program can damage another program or the operating system itself.

3 Three Simple Memory Organizations

Even in a system with no memory abstraction, the operating system must live somewhere in memory. Three simple layouts are common historically.

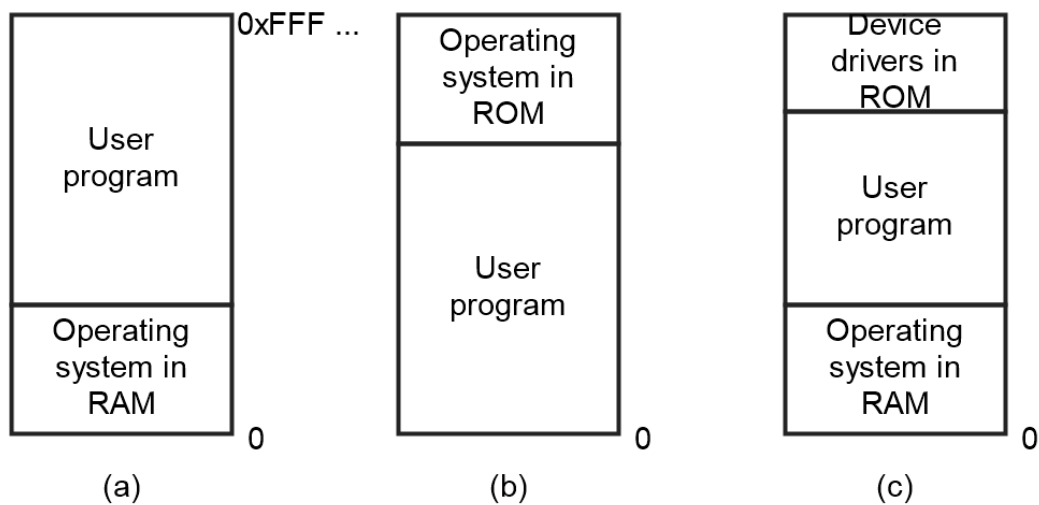


Figure 3-1. Three simple ways of organizing memory with an operating system and one user process. Other possibilities also exist.

Layout	Explanation
(a) OS in RAM at bottom	The user program is placed above the operating system. This was used historically on some mainframes and minicomputers, but a bad user program can overwrite the OS.
(b) OS in ROM at top	Operating system code is stored in read-only memory. This is safer for fixed systems because ROM cannot be overwritten by normal stores.
(c) BIOS-style split	Device drivers are in ROM, and the rest of the OS is in RAM. This was used by early PCs; the ROM part is commonly called the BIOS.

Key Observation

Layouts (a) and (c) are vulnerable because at least part of the operating system is writable RAM.

4 How a One-Program System Runs

When the machine can safely hold only one user program at a time, execution follows a simple cycle.

Step	Action
1	The user types a command.
2	The operating system loads the requested program from disk into memory.
3	The program runs using physical memory addresses.
4	The program finishes and returns control to the operating system.
5	The operating system displays a prompt and waits for the next command.
6	The next program is loaded, overwriting the previous program.

Single-Program Rule

At most one unrelated user program should occupy memory at a time.

5 Can Threads Help?

Threads can provide limited parallelism because all threads in one process are meant to share the same memory image.

Case	Effect in a no-abstraction system
Multiple threads in one process	Acceptable in principle because they intentionally share memory.
Multiple unrelated programs	Unsafe because each program expects its own independent memory.

Why Threads Are Not Enough

Threads do not give each program a private memory space. They only allow parallel activity inside one shared memory image.

Also, a system primitive enough to lack memory abstraction is unlikely to provide a rich thread abstraction.

6 Running Multiple Programs Anyway

Even without a true memory abstraction, a system can attempt to run multiple programs by using extra mechanisms.

6.1 Method 1: Swapping

Swapping

Swapping saves the entire memory image of one program to disk, then loads another program into memory.

The important point is that only one program is physically present in memory at a time. This avoids direct memory conflicts, but it is expensive because disk I/O is much slower than memory access.

Advantage	Disadvantage
Prevents two programs from overwriting each other in memory.	Saving and restoring all of memory is slow.
Does not require each program to have a private address space.	Only one program actually runs from memory at a time.

6.2 Method 2: Protection Keys

Some machines added hardware protection while still using physical addresses. Early IBM 360 systems used this approach.

IBM 360 Protection Keys

Memory was divided into 2-KB blocks. Each block had a 4-bit protection key, and the running process had a 4-bit key in the program status word (PSW).

Access Rule

$$\text{access allowed} \iff \text{PSW key} = \text{memory-block key}$$

If the keys did not match, the hardware generated a trap. Since only the operating system could change protection keys, user programs could be prevented from damaging one another.

Storage Needed for Protection Keys

For a 1-MB memory divided into 2-KB blocks:

$$1 \text{ MB} = 1024 \text{ KB}$$

$$\frac{1024 \text{ KB}}{2 \text{ KB/block}} = 512 \text{ blocks}$$

Each block needs a 4-bit key:

$$512 \times 4 = 2048 \text{ bits}$$

Since 8 bits make 1 byte:

$$\frac{2048}{8} = 256 \text{ bytes}$$

Answer

A 1-MB IBM 360-style memory needs only 256 bytes of protection-key storage.

7 Why Protection Alone Is Not Enough

Protection keys solve one problem: they stop a program from accessing memory blocks with the wrong key. However, they do not solve the deeper addressing problem.

Relocation Problem

The **relocation problem** occurs when a program contains absolute addresses that are correct only if the program is loaded at a specific physical location.

Consider two programs, each 16 KB in size.

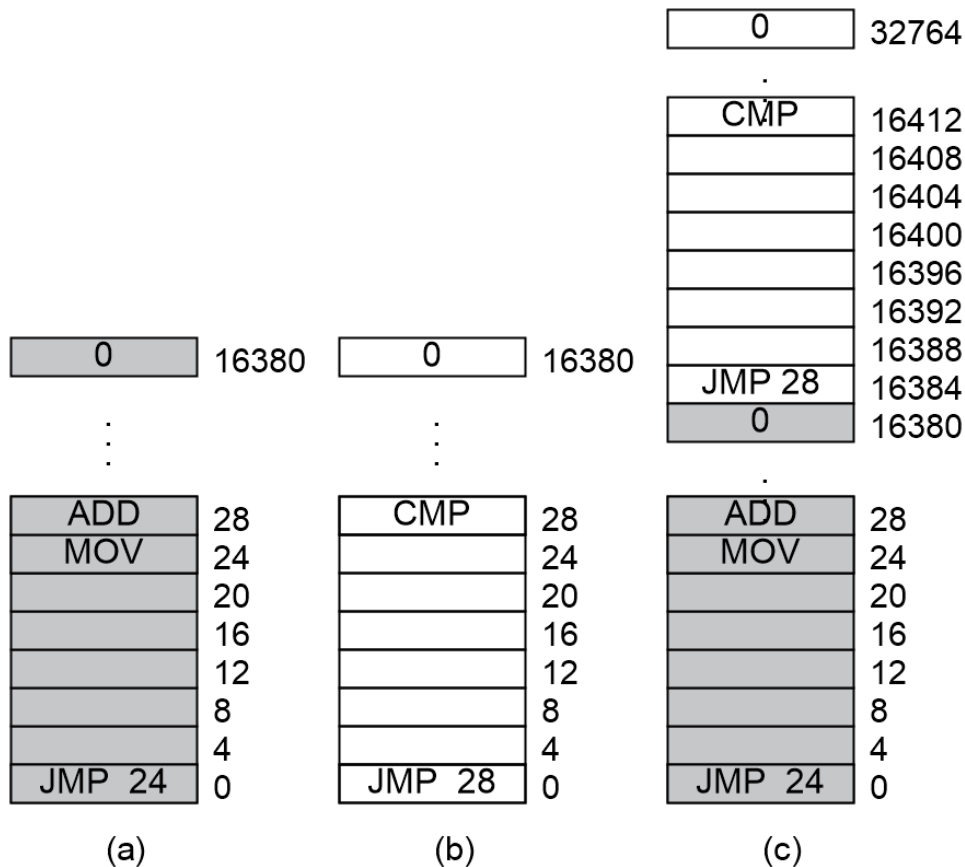


Figure 3-2. Illustration of the relocation problem. (a) A 16-KB program. (b) Another 16-KB program. (c) The two programs loaded consecutively into memory.

Program *A* is loaded first, beginning at physical address 0. Since Program *A* is 16 KB in size, Program *B* is loaded immediately after it, beginning at physical address 16,384.

Local Address vs Physical Address

Program *B* was written as if its first instruction were at address 0. So, inside Program *B*, address 28 means “the instruction 28 bytes from the start of Program *B*.”

But the hardware has no such idea of “inside Program *B*.” Without memory abstraction, address 28 means physical memory address 28.

Program *B*’s first instruction is:

```
JMP 28
```

The programmer intended this instruction to jump to Program *B*’s local address 28, where the CMP instruction is stored. Since Program *B* actually starts at physical address 16,384, Program *B*’s local address 28 is physically located at:

physical location of Program *B*'s local address 28 = 16,384 + 28 = 16,412.

However, the CPU does not automatically add 16,384. It executes the address exactly as written:

View	Meaning of JMP 28
Programmer's intention	Jump to Program <i>B</i> 's own local address 28, which is physical address 16,412.
CPU's interpretation	Jump directly to physical address 28, because no relocation or virtual memory is being used.
Actual result	Execution enters Program <i>A</i> 's memory instead of Program <i>B</i> 's CMP instruction.

Core Failure

The failure is not just lack of protection. The deeper problem is that absolute addresses are not automatically adjusted when a program is loaded somewhere other than address 0. Protection keys may stop some illegal accesses, but they do not make address 28 local to Program *B*.

8 Static Relocation

Definition

Static relocation fixes absolute addresses while the program is being loaded into memory.

If the loader places a program at base address *B*, then each relocatable address *A* inside the program is changed to:

Static Relocation Formula

$$\text{relocated physical address} = B + A$$

For Program *B*, the base address is 16,384. Therefore:

$$\text{JMP } 28 \Rightarrow \text{JMP } (16,384 + 28) \Rightarrow \text{JMP } 16412$$

Why This Helps

The program can be loaded somewhere other than address 0, and its internal address references can still point to the intended physical locations.

8.1 Limitations of Static Relocation

Static relocation is only a partial solution.

Limitation	Explanation
Slow loading	The loader must scan and modify address fields before execution.
Extra metadata	The executable must mark which words are relocatable addresses.
Constants are ambiguous	A number may be an address in one instruction but ordinary data in another.
Not flexible at runtime	Once loaded and patched, moving the program again is difficult.

The loader must relocate the 28 in:

```
JMP 28
```

But it must not relocate the 28 in:

```
MOV REGISTER1,28
```

because the second instruction may mean “put the constant value 28 into REGISTER1.”

9 Modern Relevance

Direct addressing of physical memory is no longer normal on general-purpose computers such as desktops, laptops, servers, and smartphones. These systems use stronger memory abstractions, usually including virtual address spaces and hardware memory management.

However, no memory abstraction is still practical in small embedded systems.

- **Microwave oven firmware:** the program is fixed and stored in ROM.
- **Washing machine controller:** users do not install unrelated applications.
- **Simple radio firmware:** memory use is known during design.
- **Smart card system:** the system is small and tightly controlled.

Some embedded operating systems are closer to libraries than full operating systems. The application links with the OS library and calls routines for I/O and common services. eCos is an example of this style.

10 Comparison of Approaches

Approach	Evaluation
One program at a time	Very simple, but it gives no real multiprogramming. A new program overwrites the old one.
Threads	Useful only inside one shared process image. It does not support unrelated protected programs.
Swapping	Avoids simultaneous memory conflicts by keeping one program in memory, but disk copying is slow.
Protection keys	Blocks many illegal accesses, but absolute addresses still refer to physical memory.
Static relocation	Lets programs load at different physical bases, but it needs relocation metadata and slows loading.